

AI Pilot Project

Setup & Usage Guide — grounded AI copilot for drones, rovers, and ROVs

This guide covers building and running the **ai_pilot_setup** configuration tool, and then running the Python AI pilot itself (flight control, camera, and microphone).

Scope note: this project is for navigation, monitoring, and general autonomy (search & rescue, mapping, inspection, delivery-style tasks). It does not include, and will not be extended to include, weapons, targeting, or payload-release capability.

1. What This Project Is

A local, tool-calling AI layer that sits above ArduPilot/PX4 (via MAVLink) and adds natural-language control, camera perception, and voice commands. The AI never invents facts — it can only act through a fixed set of real, verified functions (get telemetry, arm, takeoff, goto, look around, listen for a command, etc.), and every answer it gives is grounded in what those functions actually returned.

Two parts to this project:

- **setup_tool/** — a C++ command-line tool that scans your board's real hardware and writes a config file the Python app reads.
- **Python AI pilot** (`ai_pilot_perception.py` and friends) — the actual assistant: connects to your vehicle, camera, and microphone, and lets you give it natural-language commands.

Architecture

Voice/text commands → grounded AI layer (Ollama, local LLM) → tool calls only → MAVSDK/MAVLink → ArduPilot/PX4 → real vehicle. Camera and microphone feed in through the same tool-calling pattern, using local models (YOLO for vision, Whisper for speech) so nothing needs an internet connection to run once set up.

2. Requirements

On the board (Raspberry Pi, BeagleBone, Jetson, or generic Linux):

- A C++17-capable compiler (g++, already standard on Pi OS / Debian-based images)
- Python 3.9+
- Ollama installed (`curl -fsSL https://ollama.com/install.sh | sh`)
- Python packages: `mavsdk ollama opencv-python sounddevice ultralytics faster-whisper numpy`

Install Python packages with: `pip install mavsdk ollama opencv-python sounddevice ultralytics faster-whisper numpy --break-system-packages`

3. Building the Setup Tool

From the `setup_tool/` directory:

```
cd setup_tool
make
```

This produces a single executable, `ai_pilot_setup`. No extra libraries needed beyond the standard C++ library — it only reads `/proc`, `/dev`, and group membership, and writes one JSON file.

4. Running the Setup Tool

```
./ai_pilot_setup
```

It walks through several steps, in order. Everything it shows you is read from the real system — nothing is guessed.

Step 1 — Board Identification

Reads `/proc/device-tree/model` (or `/proc/cpuinfo` as a fallback) to identify Raspberry Pi, BeagleBone Black/Blue, Jetson Nano/Orin, or a generic board.

Step 2 — Permission Check

Checks whether your user is in the `dialout` (serial/USB), `video` (camera), and `audio` (microphone) groups. If any are missing, it prints the exact commands to fix it:

```
sudo usermod -a -G dialout $USER
sudo usermod -a -G video $USER
sudo usermod -a -G audio $USER
# then log out and back in
```

Step 3 — Serial Ports (flight controller connection)

Scans for `/dev/ttyACM*`, `/dev/ttyUSB*`, and `/dev/ttyS*` devices. Pick one, or choose SITL simulation (`udp://:14540`) if you don't have hardware connected yet, or enter a connection string manually (useful for Ethernet/TCP-based setups).

Step 4 — Cameras & Step 5 — Audio

Scans `/dev/video*` for cameras and runs `arecord -l` to list real microphone input devices.

Step 6 — Vehicle Type

Choose Drone, Rover, or ROV. This sets matching default safety limits, shown to you with the option to customize:

Vehicle	Key limit	Default	Notes
Drone	Max altitude	30 m	Distance 200 m, speed 8 m/s, battery 20%
Rover	Max speed	3 m/s	No altitude concept; distance 500 m, battery 15%
ROV	Max depth	20 m	Distance 100 m, speed 1.5 m/s, battery 25%

These limits are enforced in the Python flight code itself (`vehicle_tools.py`) — not just described to the AI. For example, a Rover config makes `takeoff()` refuse to run at all, at the code level.

Step 7 — AI Model Selection

A curated catalog of known-good local models is shown, each with real capability info:

Model	Vision	Tool-calling	Size	Best for
Llama 3.2 (1B)	No	Yes	~1.3 GB	Most constrained boards
Llama 3.2 (3B)	No	Yes	~2.0 GB	Recommended default (Pi 4/5)
Llama 3.2 Vision (11B)	Yes	Yes	~7.9 GB	Jetson-class GPU boards
Qwen 2.5 (3B)	No	Yes	~1.9 GB	Alternative to Llama 3B
Qwen 2.5 (7B)	No	Yes	~4.7 GB	8GB+ RAM or GPU
Qwen 2-VL (7B)	Yes	Yes	~6.0 GB	Native vision on GPU boards
Gemma 2 (2B)	No	Yes*	~1.6 GB	Lightweight alt. (verify tool-calling)
Gemma 2 (9B)	No	Yes*	~5.4 GB	More RAM available
Moondream (1.8B)	Yes	No	~1.7 GB	Vision helper only, not main brain

* Gemma 2's tool-calling is less consistent for this project's pattern — test carefully.

Selecting a model not yet installed triggers a real `ollama pull` with live progress output. Picking a model without tool-calling support (like Moondream) shows a warning and lets you choose a different one instead.

Important: no local chat model here handles audio directly. Spoken commands always go through faster-whisper separately, regardless of which model you pick.

Step 8 — Model Tuning (temperature)

Lower temperature makes the model stick closer to tool results instead of embellishing — meaningfully reduces (but does not eliminate) hallucination. "Strict/grounded" (temperature 0.1) is the recommended preset for a control system.

Step 9 — Writing Configuration

Everything above is written to `config.json` in the current directory. The Python AI pilot reads this automatically on startup.

5. Running the AI Pilot

Copy `config.json` next to the Python files (or run the setup tool from the same directory), then:

```
python3 ai_pilot_perception.py
```

If no `config.json` is found, it falls back to SITL simulation defaults so you can still test without hardware.

Example session:

```
You: what's the current status?
Pilot: Battery is at 87%, not armed, position holding at home.

You: take off to 10 meters
Pilot: I need you to confirm – should I arm and take off to 10m? (yes/no)
You: yes, confirmed

You: what do you see?
[executing: look_around({})]
Pilot: I can see one person in view.

You: listen for my command
[listening for 4.0s...]
Pilot: I heard: "return to launch point"
```

Notice it never arms or takes off without an explicit confirmation from you — that check is enforced in code, not just requested in the prompt, so the AI cannot talk its way around it.

6. Test in Simulation First

Before connecting to any real vehicle, validate everything against ArduPilot SITL (Software In The Loop) — it runs the real ArduPilot firmware against a physics simulator, no hardware required:

```
git clone --recurse-submodules https://github.com/ArduPilot/ardupilot.git
cd ardupilot
Tools/environment_install/install-prereqs-ubuntu.sh -y
. ~/.profile

# Drone:
sim_vehicle.py -v ArduCopter --console --map

# Rover:
sim_vehicle.py -v Rover --console --map

# ROV:
sim_vehicle.py -v ArduSub --console --map
```

SITL listens on `udp://:14540` by default — exactly what the setup tool's SITL option connects to.

7. Moving to Real Hardware

Once SITL testing feels solid, re-run `./ai_pilot_setup` on the actual board with the flight controller connected (USB or Ethernet/TCP). A few safety practices for first real tests:

- Keep `max_altitude_m` very low (2-3 m) for first flights.
- Test with propellers off first — verify arm and `get telemetry` respond correctly before anything spins.
- Keep `return_to_launch` and hold in mind as your escape hatch — they're always available regardless of what the AI is doing.
- Test on a tethered/controlled setup before any free flight or open-water test.

8. Porting to a New Board

Copy the `setup_tool/` folder to the new board, run `make`, then `./ai_pilot_setup` again. It detects that board's real hardware fresh and writes a new `config.json` — nothing about the tool itself needs to change between boards.

9. Safety Notes

Safety limits (altitude, depth, speed, battery) are enforced in `vehicle_tools.py` at the code level — the AI cannot override them by being convincing. If you add new capabilities, keep enforcing limits in code, not just in the prompt.

This project is scoped to navigation, monitoring, and general autonomy. It does not include, and should not be extended to include, weapons, targeting, or payload-release capability of any kind.